CAPADEV ENGINEERING CHALLENGE · TIER 2 EXPLAINED

Reporting to the National Platform

How PulseTrack exchanges data with a shared FHIR R4 server — and the three things its own API guide gets wrong about it, each of which fails silently.

STANDARD

FHIR R4 — `Patient` and `Observation` only.

SHARED WITH

Every other candidate. Reads are open; writes are not.

MOVES

Both ways. 14 resources pushed, 180 observations pulled.

UNDO

None. `DELETE` is disabled — every write is permanent.

THE TASK

A clinic that has to report to somebody else's database

Tier 1 built a clinic that keeps its own records. Tier 2 connects it to a national health data platform — a server the clinic does not own, cannot delete from, and shares with every other organisation reporting to it.

That last sentence is the whole engineering problem. A private database is forgiving: a mistake can be corrected, a duplicate removed, a bad write rolled back. None of that is true here. The server is a **HAPI FHIR R4** instance standing in for a national platform, and it enforces the constraints a real one would.

Reads

Open to everyone

Any organisation can search any record. A resource being *visible* says nothing about whether it may be modified.

Writes

Only what you created

The server stamps every resource with its creator's tag. Attempting to change somebody else's earns a `403`, permanently.

Deletes

Disabled entirely

There is no undo. A duplicate created by a careless sync stays on the national platform for good.

Budget

120 requests a minute

Shared across everything the deployment does. Handling `429` is named in the brief as part of the exercise.

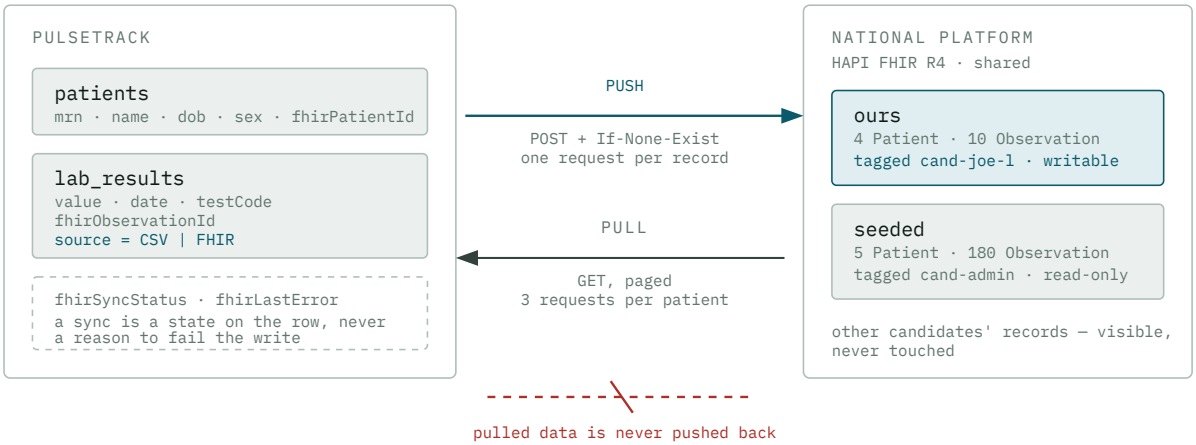
The brief asks for four things: **push** patients and lab results as they are entered, **pull** the platform's own historical data for five seeded patients, **handle the realities** of an external API — authentication, failures, retries, and not importing the same thing twice — and **document the data flow**. All four are built.

> THE RULE EVERYTHING ELSE FOLLOWS FROM
>
> Deletes being disabled is not a detail to work around; it is the constraint that determines the design. It is why every write is a conditional create rather than a plain one, why ownership is verified from the server's own response rather than inferred, and why the mapping and reconciliation logic is written as pure functions — those are the only parts of this integration that can be exercised freely, because testing them costs nothing and cannot leave anything behind.

THE SHAPE

Two directions, one table

Data moves both ways, and the two directions are asymmetric in a way that matters. What we push, we own and may revise. What we pull belongs to the platform and is read-only to us. Both end up in the same two tables, which is what lets a clinician see imported history and locally-entered results on one chart.



Both directions land in the same two tables. **The `source` column is what makes that safe** — it is how a chart can mix the two, and how the push queue knows to exclude the 180 rows that came from the platform in the first place.

FINDING 1 · OWNERSHIP

The API guide's own example would have bound us to a stranger's record

This is the finding that shaped the implementation, and it was found before a line of client code was written — by reading the server instead of trusting the guide.

FHIR's idiomatic way to make a create idempotent is a **conditional create**: `POST` the resource with an `If-None-Exist` header carrying a search query. If the search matches, the server returns the existing record instead of making a second one. The guide gives this example:

```
POST /Patient
If-None-Exist: identifier=https://challenge.capadev.dev/mrn|MRN-1001
```

`MRN-1001` is the medical record number printed in the supplied CSV template — so every candidate who seeds their database from the sample data creates a patient with it. Since writes are permanent, the sensible first move was a read-only query rather than a write:

```
READ-ONLY PROBE, BEFORE ANY WRITE
$ GET /Patient?identifier=https://challenge.capadev.dev/mrn|MRN-1001

total = 5
  id=189  tag=cand-jihane-l
  id=265  tag=cand-maryamhmayed-l
  id=278  tag=cand-marwa-l
  id=340  tag=cand-adham-l
  id=360  tag=cand-khalils-l
```

Five other organisations already hold that MRN. The guide's header matches all five, and a conditional create whose search matches more than one resource returns `412`.

But the *worse* outcome is the one that nearly happened to everybody who follows the guide literally. Had the search matched exactly **one** foreign resource — which is only a question of how many other candidates had reached that MRN first — the server would have returned somebody else's resource id, we would have stored it as our own, and every later update to that patient would have earned a permanent `403`. A failure that looks exactly like a bug in our own code.

AS THE GUIDE WRITES IT

```
If-None-Exist:
  identifier=…|MRN-1001
```

Searches **every** organisation's records, because reads are open. Matches five foreign resources → `412`. Match exactly one and it silently binds us to a record we can never write to.

AS PULSETRACK SENDS IT

```
If-None-Exist:
  identifier=…|MRN-1001
  &_tag=…/tags|cand-joe-l
```

Scopes idempotency to **our own** records. `_tag` is a standard search parameter, and conditional creates accept arbitrary search parameters, so this is idiomatic FHIR rather than a workaround.

Whether HAPI honours `_tag` *inside* that header was the open question. The five-way collision turned it into a clean experiment rather than a hopeful one: if `_tag` were ignored, the search would still match five and the server would answer `412`. It answered `201`.

```
THE FIRST WRITE, AND THE SECOND

$ POST /Patient   (tag-scoped)
HTTP/1.1 201 Created
Location: http://hapi:8080/fhir/Patient/816/_history/1
"tag": [{ "system": ".../tags", "code": "cand-joe-l" }]

$ POST /Patient   (identical request)
HTTP/1.1 200 OK
Location: http://hapi:8080/fhir/Patient/816/_history/1
```

`201` then `200`, same id, no duplicate. Two open questions closed in about four minutes, and the answer changed the design rather than confirming it.

Belt and braces

Scoping the search is not quite enough on its own. After any create-or-match, PulseTrack reads the **server's own** **`meta.tag`** on the response and only treats the resource as writable if our organisation's tag is present. Ownership is asked of the server, never inferred from our request having succeeded — because the cost of being wrong is a write that cannot be undone.

> TWO SMALLER TRAPS IN THE SAME TWO RESPONSES
>
> The `Location` header carries the server's **internal container hostname**, `hapi:8080`, so it must be parsed and never fetched. And the resource id is the segment **before** `_history`, not the last one — reading the last segment stores the version number `1` as the resource id, and every later write goes to a record that does not exist.

FINDING 2 · PAGINATION

The guide says to follow the `next` link. The `next` link does not work

The platform holds twelve months of monthly results for five seeded patients. Fetching them means following a paged search — and the guide's instruction to *"follow the bundle's `next` link"* cannot be carried out as written.

The failure this prevents is a silent one. Fetching `hapi:8080` from outside the cluster simply fails, so an import that follows the link verbatim stops after its first page — and reports success. A truncated import is worse than a failed one, because nothing about it looks wrong.

And a page size chosen so the code actually runs

The guide suggests `_count=50`. Each seeded patient has exactly **36** observations — so at 50, every patient returns in a single page, no `next` link is ever emitted, and the pagination code never executes once. It would be an integration that *looks* paginated and has never paginated.

PulseTrack requests `_count=20`, which pages every patient twice — 20 then 16. The per-patient import report prints the page count for exactly this reason:

```
THE SEEDED IMPORT, RUN AGAINST THE LIVE SERVER

all five: found=true created=true updated=0 merged=0 conflicts=0

MRN-2001: imported=36  pages=2  truncated=false  skipped=[]  (2360ms)
MRN-2002: imported=36  pages=2  truncated=false  skipped=[]  (2232ms)
MRN-2003: imported=36  pages=2  truncated=false  skipped=[]  (2569ms)
MRN-2004: imported=36  pages=2  truncated=false  skipped=[]  (2560ms)
MRN-2005: imported=36  pages=2  truncated=false  skipped=[]  (2150ms)

labs:     180 source=FHIR  ·  10 source=CSV
patients: 5 EXTERNAL_SEED  ·  4 OWNED
```

Three further rules govern the walk, each guarding a way a cursor loop against someone else's server can misbehave. It stops if a cursor points back at a page already read. It caps the number of pages. And it **reports truncation** rather than hiding it. Loop control depends

on the presence of a `next` link and nothing else — `bundle.total` is present on an unpaged search and *absent* on a paged one, so counting against it never terminates.

THE REQUIREMENT

Running it twice changes nothing

"Not re-importing the same data twice" is one of the brief's four Tier 2 requirements. On a server with no delete, it is also the requirement with no second chance.

DO THIS TWICE	GUARANTEE	HOW
Push a patient	No duplicate remote `Patient`	Tag-scoped `If-None-Exist` on the MRN identifier. The second call returns `200` and the same id.
Push a lab result	No duplicate remote `Observation`	Keyed on the local row's **cuid**, not on anything shared. An MRN is common vocabulary; a cuid is ours alone, so the match is exact even though reads are open to everyone.
Retry after a lost response	Still no duplicate	Exactly why every write is a conditional create rather than a plain `POST`: the retry's own search finds whatever the lost response created.
Import the seeded data	No duplicate local rows, and no writes at all	Patients upserted by unique `mrn`; observations matched by unique `fhirObservationId`, then by `(patient, date, test)`.
Fail mid-sync	No local data loss	The local commit is a separate step from the sync, which only ever records a status on the row.

Measured rather than asserted — the resource counts on the platform, taken before and after a complete re-push:

```
RE-PUSH, COUNTED AT THE SOURCE
BEFORE  Patients=4   Observations=10

  batch: {"patientsPushed":4,"resultsPushed":0,"failed":0,
          "more":false,"remaining":0}

AFTER   Patients=4   Observations=10

re-import, each of the five patients
  imported=0  updated=0  unchanged=36  merged=0
```

The collision that makes this harder than it looks

Two unique constraints can disagree about the same observation. One says *one local row per resource on the platform* (`fhirObservationId`). The other says *one local row per measurement* (`patient + date + test`). They collide whenever a lab result already entered from a CSV describes the measurement the platform is now reporting — and letting the second constraint throw would fail an entire import over one row that is not even wrong.

The decision is made by a pure function, which is the point: every branch of it is tested in milliseconds instead of by uploading a CSV and then importing against a server whose writes are permanent. Three outcomes:

`imported`	`updated`	`merged`
New to us	**Already linked**	**Already held locally**
The platform holds a measurement we have never seen. Written with `source = FHIR`.	The platform owns this row and is the authority on it, so a revised value is taken. Identical values are not rewritten at all.	The link is attached and **the stored value is not touched**. Where the two disagree, that is counted and shown.

WHY A MERGE NEVER OVERWRITES

Silently editing a clinician's record because a remote resource happens to share a date is how a clinical system stops being trustworthy. Resolving the disagreement automatically would mean choosing a winner without a clinician, which is not ours to choose — so PulseTrack shows the conflict instead. It is also the same rule a re-uploaded CSV row already follows: a changed value is reported, never silently applied.

THE REALITIES

Which failures are worth retrying, and which are facts

The distinction that governs the whole error layer is **transient versus terminal**. A `403` is not a failure to retry — it is a permanent fact about ownership, and retrying it spends a 120-per-minute budget re-learning it. Treating every non-2xx as "try again" is how a sync queue stops draining.

STATUS	RETRIED	WHAT IT MEANS, AND WHAT THE CLINICIAN SEES
400 · 422	no	Our resource is malformed. The server's own `diagnostics` is shown; retrying would send the same bad resource again.
401	no	Surfaced as *"check `FHIR_API_KEY`"* — named as the thing to fix, not as "unauthorized".
403	no	Recorded as a **state**, not an error, and excluded from the queue so it can actually drain. Listed on the integration page with the server's explanation.
404	no	The linked resource is gone.
405	no	We issued a disabled operation — a delete, a conditional update, an update-as-create. A programming error, and one worth failing loudly.
412	no	A conditional create matched more than one resource. Surfaced for inspection — this is the status the guide's own example produces.
429	yes	Honours the server's `Retry-After` when it sends one, else exponential backoff **with jitter**.
5xx · timeout	yes	Up to three attempts with backoff. The request may have been received, which is why every write is idempotent.

Verified by breaking it deliberately — pointing the client at a wrong key, then at a host that does not resolve, then back. The *timings* are the evidence, not the messages:

```
FAILURE PATHS, RUN AGAINST THE REAL CLIENT

== wrong API key -> 401 ==
  error : "The national platform rejected our credentials.
           Check FHIR_API_KEY."
  stored: FAILED | Missing or invalid X-API-Key header.
  link  : 816 kept | 1218ms  <- one attempt, no retry

== unreachable host -> network ==
  error : "Could not reach the national platform."
  link  : 817 kept | 4302ms  <- three attempts, backoff

== recovery on the real server ==
  result: {"ok":true,"fhirId":"818","created":false}
  stored: SYNCED | no error
```

1.2 seconds is one attempt. 4.3 seconds is three with backoff. Both kept the resource link, so recovery was a matter of the platform coming back rather than of re-establishing anything.

Rate limiting is handled before it happens

Reacting to a `429` is necessary but not sufficient. A seeded import is three requests per patient and a push is one per record, so importing and then syncing a CSV can put a few hundred requests through in under a minute. An unthrottled client earns the limit rather than avoiding it — and every retry then lands the run further behind. PulseTrack holds a sliding window at **100 requests a minute** against the documented 120, with at most four in flight, leaving the margin that absorbs a burst of retries instead of converting it into more of them.

> THE INTEGRATION CAN FAIL WITHOUT THE CLINIC FAILING
>
> Patient CRUD and the CSV importer reach the platform through a single seam. Neither imports the FHIR client, neither handles a FHIR error, and neither can be broken by one. The local write always commits first, so an outage never rolls back a clinical record and never blocks data entry — it leaves a visible sync state on the row instead. With no FHIR key configured at all, the whole of Tier 1 still runs and the integration page says plainly which variables are missing.

THE DEPLOYMENT

Why the work is done in batches the browser drives

A Vercel Hobby function stops at 60 seconds, and both directions of this integration are naturally larger than that. Doing either in one request is the shape that works on a laptop and times out on the deployed URL — in front of the person evaluating it, with no explanation.

push	pull	exception
A bounded batch, looped	**One patient per request**	**A patient syncs inline**
One request per record, and a CSV can hold thousands of rows. `/api/fhir/sync` pushes a fixed slice and reports whether more remain; the browser calls it until the queue drains.	Each patient is three requests and 36 rows. Per-patient means a patient that fails does not take the other four with it, and the report can say what happened to *that* patient.	Bounded at six seconds, because the brief asks for a patient to sync *when they are created*. If the budget lapses the record is already queued, so nothing is lost.

The loops also give the interface something honest to show — a count going down, a row per patient — rather than a spinner of unknown length. Each stops on three explicit conditions: the queue drains, the server reports it made no progress, or a hard cap is reached. A retry loop against an external service needs a stop condition that does not depend on that service behaving.

OUR OWN FINDING · COST

Idempotent is not the same as cheap

Both directions were idempotent on the first attempt. Both were also quietly heading for a function timeout, and neither would have failed a test.

The re-runs produced perfect results — identical resource counts, zero new rows. What they did not produce was acceptable timings, and that only became visible because the runs were timed rather than merely checked.

PUSH · 14 RECORDS

~~12.8s~~ **3.2s**

Pushed one at a time, so the concurrency cap of four was decorative — nothing ever ran concurrently. A fifth of the function budget spent waiting.

RE-IMPORT · PER PATIENT

~~10s~~ **2.1s**

180 unchanged rows written back one at a time, inside a transaction whose own timeout is 20 seconds. Linked rows are now compared before being written.

The second fix has a side benefit worth naming: because an unchanged row is no longer rewritten, **"updated" now means something a clinician can read** — the platform revised this result — rather than being a count of rows we happened to touch.

And fixing the first one introduced a real bug

Making the push concurrent created a race that the sequential version could not have. An `Observation` whose patient is not yet linked pushes that patient itself — so two results for the same unlinked patient issue two conditional creates whose *searches both complete before either insert lands*. Both find nothing. Both create. That is a duplicate `Patient` on a server where `DELETE` is disabled.

The fix is ordering rather than locking: every referenced patient is linked **before** any observation is sent, which removes the race by construction rather than relying on losing it. Re-verified afterwards — still 4 patients and 10 observations under our tag, across three further re-pushes.

> THE GENERAL LESSON
>
> A correct result and a survivable one are two different tests. Both sync directions passed every correctness check while sitting at a fraction of a function budget that real data volume would have exhausted. Timing a run that has already passed is a cheap check that nothing else performs.

EVIDENCE

What was actually checked, and how

Every number in this document is output from a run, not an estimate. The integration was exercised three ways, because each catches something the others do not.

CHECKED BY	WHAT IT ESTABLISHED
264 unit tests	The mapping in both directions, that `400/401/403/404/405/412/422` are never retried while `429/5xx/timeout` are, `Retry-After` handling, the throttle's ceiling and concurrency cap, and every branch of the import's two-constraint reconciliation — including that a locally-held value is never overwritten. These carry more weight than usual here: the mappers and the reconciler are the only parts of this integration that can be exercised without writing to a server that has no undo.
Runs against the live server	The conditional-create behaviour, the tag scoping, pagination over two real pages per patient, 180 observations imported, identical counts across repeated pushes and imports, and the three failure paths with their timings.
A headless browser	Both loops driven through their own buttons — five `200` s from the import endpoint, per-patient progress — the pulled data confirmed *on the rendered charts* rather than inferred from row counts, no layout overflow at 1280px or 375px, and the API key absent from the page HTML.
A secret scan	After a production build: the key appears in **0** files under the client bundle and **0** commits in git history. The client module and its config carry `server-only` , so an import from a browser component is a build error rather than a leaked credential.

ONE PROBE LIED, AGAIN

The first browser test of the push button reported that no API call had been made. It had — the probe closed **12.7 seconds** into a request that took 12.8, and the server log settled it in one line. A probe that stops before the thing it is measuring proves nothing, and a green-looking check that never reached the code under test is worse than no check at all.

HONESTY

What this integration does not do

Each of these is a deliberate trade rather than an oversight, and each has a cost that is worth stating rather than discovering on a call.

- **The sync is inline and queued, not an outbox with a background worker.** A real system would commit an event alongside the local write and drain it from a job runner, which survives a deployment mid-sync and needs no browser tab open. That needs infrastructure this submission does not have, and it would give the clinician nothing to watch.

- **A patient's name is split by guessing.** We store one `fullName` because the brief asks for one; FHIR wants a family name and given names. The last whitespace-separated token becomes the family name, which is wrong for "van der Berg" and for cultures that put the family name first. The guess is made in one place, where it can be seen and corrected.
- **Contact details are deliberately not sent.** Email and phone would map cleanly to FHIR `telecom`, and they are omitted anyway: the server is shared and reads are open, so every field pushed is readable by every other organisation on it. The integration needs identity and demographics to link observations; it does not need a way to contact the patient.
- **A local and remote disagreement is shown, not resolved.** When an imported measurement contradicts one already on file, both remain and the conflict is counted. Picking a winner is a clinical judgement.
- **Deleting a patient locally leaves an orphan on the platform.** The server disables `DELETE`, so there is nothing to call. This is documented rather than hidden.
- **The import has not yet run from the deployed URL.** Every measurement here is local. The per-patient timings suggest the 60-second ceiling is comfortable, but that is a prediction, and this project's own history is a list of predictions that a measurement disproved.

THE CALL

Questions this design invites

Why not use a FHIR client library?

Because an SDK hides precisely the behaviour being evaluated. Conditional creates, ownership tags, pagination cursors and `OperationOutcome` parsing are the exercise; delegating them would mean answering questions about a dependency rather than about a decision. The whole client is a few hundred lines and every rule in it can be explained.

What happens if the server stops honouring `_tag` in `If-None-Exist`?

The conditional create would match a foreign resource or nothing, and would create a second copy of each patient rather than failing loudly. That is exactly why `meta.tag` is verified on every response instead of being trusted once: a resource that comes back without our tag is recorded as not ours and never written to.

Why is a lab result keyed on a cuid rather than on the measurement itself?

Because the identifier has to be unique across every organisation on a shared server, and a natural key is not. Patient plus date plus test code collides with any other candidate reporting the same seeded patient; a cuid is unique by construction. It is the same reasoning that made the MRN unsafe as an idempotency key.

A push failed. What happened to the lab result?

It is in the record, with a visible sync state and the server's own explanation. Local writes commit first and the sync is a separate recorded step, so a failure never rolls anything back. Transient failures re-queue; a `403` is listed as refused rather than retried, because the platform will keep giving the same answer.

How do you know the re-import really writes nothing?

Two ways, deliberately independent. The reconciliation function is pure and a test asserts that 36 already-linked observations produce zero creates, zero links and zero updates. And the live re-run reports `unchanged=36` per patient at 2.1 seconds — the timing being the corroboration, since 180 writes could not complete that fast.

What would you change with more time?

An outbox and a background worker instead of browser-driven batches; structured patient names rather than one field; and a reconciliation view that lets a clinician resolve a local-versus-remote disagreement in the interface rather than only seeing that one exists.

PulseTrack · Tier 2 explained Companion to 02-project-overview.html and the Tier 1 acceptance checklist

Every figure is measured output Fabricated data only